LABORATORY

LAB 04

Fundamentals of Analog Read

Reading Analog Signals with EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction . . . 2
1.1 Lab Overview . . . 2
1.2 Objectives . . . 2
2 Background . . . 2
2.1 Analog Signals and Digital Representation . . . 2
2.2 Analog-to-Digital Conversion . . . 2
2.3 ADC Resolution . . . 3
2.4 LDR Photoresistor . . . 3
2.5 Voltage Divider with LDR . . . 3
3 Hardware Setup . . . 4
3.1 Required Hardware . . . 4
3.2 ADC Pin Mapping . . . 4
3.3 LDR Circuit Connection . . . 4
4 EduFramework API . . . 5
4.1 `analogRead()` . . . 5
5 Lab Exercise . . . 5
5.1 Requirements . . . 5
5.2 Program . . . 5
5.3 Verification . . . 6
6 Extension . . . 7
6.1 Reading Voltage with `analogReadMilliVolts()` . . . 7
6.2 Non-blocking ADC Read . . . 7
7 References . . . 8

1 Introduction

1.1 Lab Overview

In embedded systems, many physical quantities do not exist only in two logic states such as `HIGH` and `LOW`, but vary continuously with environmental conditions. To allow a microcontroller to process these signals, an analog voltage must be converted into a digital value through an Analog-to-Digital Converter (ADC).

This lab introduces analog signal reading using the ADC and the EduFramework `analogRead()` API. An LDR photoresistor is connected in a voltage divider to produce a voltage that changes with lighting conditions. The acquired ADC value is transmitted to the Serial Monitor for observation using the knowledge introduced in the previous lab.

The exercise focuses on raw ADC values rather than conversion to lux. This allows the relationship between analog signals, sampling, ADC resolution, and digital values to be observed directly on the hardware.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Distinguish between analog signals and digital representations used in embedded systems.
2. Explain the basic ADC concepts of sampling, quantization, and resolution.
3. Describe how a photoresistor and a voltage divider produce a voltage signal that can be applied to an ADC input.
4. Use `analogRead()` to read a 12-bit ADC value through EduFramework.
5. Build and verify an application that monitors changes in light level on the Serial Monitor.

2 Background

2.1 Analog Signals and Digital Representation

An analog signal can vary continuously in time and amplitude. In contrast, a digital system processes data as discrete values. Therefore, when a physical quantity is converted into an analog voltage, the microcontroller requires a conversion process to represent that voltage as digital data.

Sampling is the process of converting a continuous-time signal into a sequence of discrete samples [1]. The ADC plays an important role in this data acquisition process: the input voltage is sampled and the result is represented by a numerical code that the program can process.

2.2 Analog-to-Digital Conversion

The analog-to-digital conversion process can be described using two basic concepts: sampling and quantization.

Sampling determines the value of an analog signal at discrete points in time. The time interval between two consecutive samples is called the sampling period, commonly denoted by T_s. The sampling frequency F_s is related by:

$$F_s = \frac{1}{T_s}$$

After sampling, the amplitude of each sample must be represented using a finite number of levels. The process of assigning an analog value to one of the available digital levels is called quantization. Because the number of representable levels is finite, the resulting digital value is only a discrete representation of the input voltage.
Quantization error is the difference between the actual analog value and its quantized representation. When rounding to the nearest quantization level, this error typically does not exceed half a quantization step (0.5 LSB – Least Significant Bit). Increasing ADC resolution reduces the size of each quantization step. [1] .

2.3 ADC Resolution

Resolution defines the number of bits used to represent the conversion result. For an ADC with a resolution of N bits, the number of representable codes is:

$$2^N$$

The S32K1xx integrates ADCs with a resolution of up to 12 bits [5] . EduFramework currently configures the analog read path in 12-bit mode [7] . Therefore:

$$2^{12} = 4096$$

The raw ADC result contains 4096 codes, from `0` to `4095` .

Resolution	Number of Codes	Raw Value Range
12-bit	`4096`	`0` to `4095`

Table 4.2.1. Representation of a 12-bit ADC Result in EduFramework

The values `0` and `4095` are the codes at the two ends of the conversion range, not units of voltage or light intensity. The relationship between an ADC code and voltage depends on the ADC reference voltage. In this lab, EduFramework configures the resolution at a fixed 12 bits, so the application does not need to change the resolution before reading.

2.4 LDR Photoresistor

A photoresistor, commonly called an LDR (Light-Dependent Resistor), is a component whose resistance changes with the amount of incident light [2] . Therefore, an LDR can be used as a light-sensitive element in applications that need to detect relative changes in lighting conditions.
The ADC does not directly measure the resistance of the LDR. To convert the resistance change into a quantity that the ADC can read, the LDR is combined with a fixed resistor to form a voltage divider [2] .
This lab uses the LDR to observe the trend of changing light conditions and does not perform calibration in lux. The ADC value depends on the characteristics of the LDR, the fixed resistor, the supply voltage, and the actual lighting conditions.

2.5 Voltage Divider with LDR

The voltage divider used in this exercise consists of the LDR on the supply side and a fixed `10 kΩ` resistor on the GND side. The midpoint between the two components is connected to the ADC input. With this configuration, the voltage at the measurement point can be expressed by the voltage-divider relationship:

$$V_{\mathrm{ADC}} = V_{\mathrm{CC}} \times \left(\frac{R_{\mathrm{fixed}}}{R_{\mathrm{LDR}} + R_{\mathrm{fixed}}} \right)$$

When increased illumination causes the LDR resistance to decrease, the midpoint voltage increases, and the ADC therefore produces a larger digital value. When the LDR is covered and the light level decreases, the opposite trend occurs. This relationship was verified on the circuit used in the exercise.

Note

The ADC value represents the voltage at the midpoint of the voltage divider. It is not a measurement of light intensity in lux unless the sensor has been calibrated.

3 Hardware Setup

3.1 Required Hardware

This lab uses an LDR and a `10 kΩ` resistor to create an external voltage divider. The midpoint voltage is applied to an ADC channel of the S32K144.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
LDR	Photoresistor used to produce a signal that changes with light level.
10 kΩ Resistor	Fixed resistor used to form a voltage divider with the LDR.
Breadboard	Used to assemble the voltage-divider circuit.
Jumper Wires	Connect the power, GND, and ADC signal between the board and the external circuit.
USB Cable	Connects the board to the computer for power, programming, and Serial Monitor communication.

Table 4.3.1. Hardware Used in the Exercise

3.2 ADC Pin Mapping

EduFramework currently supports two Logical Pins for Analog Input: `ADC0_SE12` and `ADC0_SE13` [7] . This exercise uses `ADC0_SE12` , which is mapped to `PTC14` on the S32K144.

Component	Logical Pin	MCU Pin	Function
LDR Voltage-Divider Output	`ADC0_SE12`	`PTC14`	Analog Input / ADC0 Channel 12

Table 4.3.2. Analog Input Mapping Used in the Exercise

3.3 LDR Circuit Connection

The LDR is connected from the circuit supply to the ADC measurement point. The `10 kΩ` resistor is connected from the ADC measurement point to GND. The midpoint is connected to `ADC0_SE12` (`PTC14`). The board and the external circuit must share a common GND.

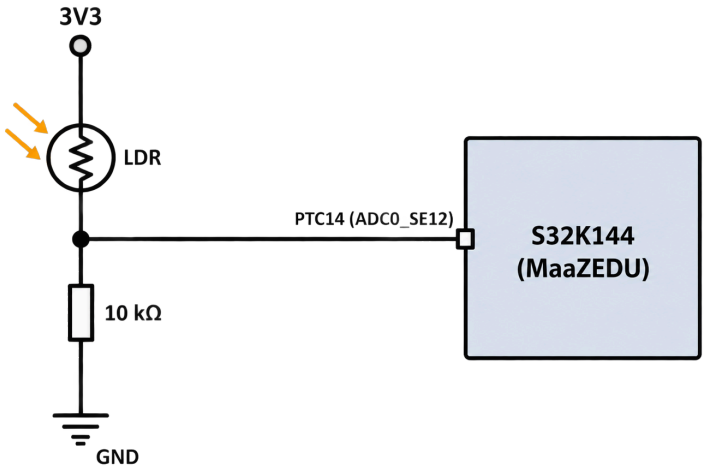


Figure 4.3.1. Connection Diagram of the LDR and 10 kΩ Resistor to the Analog Input

With the connection verified on the hardware, increasing the light level increases the ADC value, while covering the LDR decreases the ADC value. A fixed threshold is not required in this lab because the actual value varies with the component and environmental conditions.

4 EduFramework API

4.1 `analogRead()`

`analogRead()` performs one ADC conversion on the specified Analog Logical Pin and waits until the result is available. In the current version of EduFramework, the supported inputs are `ADC0_SE12` and `ADC0_SE13` [7].

analogRead

Reads an Analog Input and returns the raw ADC result.

Syntax

```
analogRead(pin);
```

Parameters

Parameter	Type / accepted value	Description
pin	Analog Logical Pin	Analog pin to read. The current version supports `ADC0_SE12` and `ADC0_SE13`.

Return value

Raw ADC value when the conversion succeeds; `-1` if the pin is invalid or initialization/conversion fails.

Example:

```
int value = analogRead(ADC0_SE12);
```

5 Lab Exercise

5.1 Requirements

Build a program that reads the LDR through `ADC0_SE12` and continuously displays the raw ADC value on the Serial Monitor. The program uses a baud rate of `9600` and updates the result approximately every `500 ms`. Change the lighting conditions by illuminating the LDR more strongly or covering it to observe the trend in the ADC value.

5.2 Program

In `src/main.c`, implement the program that was verified on the hardware as follows:

```
#include "Arduino.h"

int main(void)
{
    uint16_t lightValue = 0U;

    setup();

    Serial1_begin(9600U);

    Serial1_println("EduFramework Analog Read");
    Serial1_println("LDR monitoring started.");

    while (1)
    {
        lightValue = analogRead(ADC0_SE12);

        Serial1_print("ADC: ");
        Serial1_printlnInt(lightValue);

        delay(500U);
    }

    return 0;
}
```

Listing 4.5.1. Program for Reading the LDR through Analog Input

`setup()` initializes the fundamental components of EduFramework. Then, `Serial1_begin(9600U)` initializes the Serial Monitor channel using the configuration introduced in Lab 03.

In the main loop, `analogRead(ADC0_SE12)` performs one ADC read and stores the result in `lightValue`. This value is sent to the Serial Monitor using the Serial APIs introduced previously. `delay(500U)` creates an interval between two consecutive updates, making the changes slow enough to observe.

The application flow can be summarized as:

`Light` → `LDR + voltage divider` → `ADC0_SE12` → `analogRead()` → `Serial1` → `Serial Monitor`.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board, then open the Serial Monitor at a baud rate of `9600`.

Observe the ADC value under three conditions: ambient light, stronger illumination on the LDR, and the LDR covered. A specific ADC value is not required; the objective is to verify a stable trend as the lighting conditions change.

The Serial Monitor output has the following form:

```
EduFramework Analog Read
LDR monitoring started.
ADC: ...
ADC: ...
ADC: ...
```

EXPECTED RESULT

The Serial Monitor updates one ADC value approximately every `500 ms`. With the voltage-divider circuit used in the exercise, the value increases when the LDR is illuminated more strongly and decreases when the LDR is covered. The value remains within the 12-bit ADC code range from `0` to `4095`.

6 Extension

The main exercise uses `analogRead()` to focus on the Analog Input concept and raw ADC values. EduFramework also provides an API for converting the result to millivolts and a group of APIs for non-blocking ADC reads [7].

6.1 Reading Voltage with `analogReadMilliVolts()`

`analogReadMilliVolts()` performs an analog read and converts the raw ADC result to millivolts according to the reference voltage configured in EduFramework.

`analogReadMilliVolts`

Reads an Analog Input and returns the value converted to millivolts.

Syntax

```
analogReadMilliVolts(pin);
```

Parameters

Parameter	Type / accepted value	Description
`pin`	Analog Logical Pin	Analog pin to read, for example `ADC0_SE12`.

Return value

Voltage value in millivolts when successful; `-1` if the operation fails.

Example:

```
int voltageMv = analogReadMilliVolts(ADC0_SE12);
```

In the current version of EduFramework, this conversion uses the default reference value of `5000 mV` configured in `wiring_analog.c` [7]. Therefore, when the result is used as an accurate voltage measurement, the actual reference voltage of the system must match the configuration used in the framework.

Extension: Replace the raw ADC value on the Serial Monitor with the value in millivolts and observe the relationship between the two representations as the lighting conditions change.

6.2 Non-blocking ADC Read

`analogRead()` is a blocking API: the function starts the conversion and returns only after the result is available. For an application that needs to continue other processing while waiting for the ADC, EduFramework provides three APIs:

- `analogStart(pin)` starts one conversion and returns immediately.
- `analogAvailable()` checks whether the result is available.
- `analogGetResult()` retrieves the result of the completed conversion.

The usage flow can be represented by the following short example:

```
analogStart(ADC0_SE12);

/* Other application processing */

if (0U != analogAvailable())
{
    value = analogGetResult();
}
```

Listing 4.6.1. Principle of Non-blocking ADC Reading

Extension: Rewrite the LDR reading section of the exercise using `analogStart()` → `analogAvailable()` → `analogGetResult()` without using `analogRead()`. After receiving the result, display the value on the Serial Monitor and start the next conversion. Do not change the hardware circuit.

7 References

[1] Iowa State University, *Chapter 6: Sampling Theory* , EE/CprE/HSSE Lab.

[2] Carnegie Mellon University, *Reading a Photoresistor: How Light Is It?* , 60-223 Introduction to Physical Computing.

[3] Arduino, *analogRead()* , Arduino Language Reference.

[4] Arduino, *analogReadResolution()* , Arduino Language Reference.

[5] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.

[6] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[7] EduFramework, *Analog API* , EduFramework Source Code.